Selective Replay Under Incomplete Dependency Information

An exploratory benchmark of correctness and audit cost

Matthew Looney - Magnexis

ORCID: https://orcid.org/0009-0007-0025-8000

October 21, 2026 - Draft v0.1 - Not peer reviewed

Abstract

Reusing intermediate results can reduce the work required to rerun a computational workflow, but its correctness depends on identifying the consequences of changed inputs. We investigate the hypothesis that sparse, fresh-execution audits can recover much of the correctness lost through incomplete dependency information while retaining a substantial work reduction. A dependency-free Python benchmark compares full replay, direct-only invalidation, transitive invalidation, and two sampled-audit policies across 480 synthetic scenarios and 2,400 policy evaluations. With complete dependencies, transitive invalidation matches full replay in every pilot scenario and executes a mean 36.3% of its task count. At 5% independent edge omission, it produces an incorrect result in 74 of 160 scenarios; a 10% audit reduces this to 46 while increasing mean work to 70.7% of full replay. At 20% omission, the same audit leaves 47 incorrect runs and uses 88.0% of full work. Thus, sparse auditing mitigates errors in this model but does not establish reliable replay at low cost. We provide a restricted correctness argument, executable artifacts, and a prospective validation protocol. This is an exploratory systems study, not a new proof of general workflow correctness or evidence about deployed AI agents.

1. Motivation and research question

Developer tools, build pipelines, and local automation frequently repeat work after a small edit. Reusing previous task outputs is attractive on machines with limited compute. However, an implementation can save work by accidentally preserving stale output. A useful replay policy must therefore be judged jointly on correctness and cost. Measuring work savings alone rewards unsound policies.

Our question is: when a workflow's declared dependency structure omits some actual reads, how much correctness can a sampled fresh-execution audit recover, and what work does that recovery consume? A future application could be a local automation runtime that records dependencies and supports selective reruns. The present implementation is a deterministic simulator; it neither executes arbitrary projects nor integrates an LLM.

The unresolved hypothesis is that there exist practically useful workload classes for which sparse auditing reduces the incorrect-run rate by at least half while using no more than 75% of full-replay task work. These thresholds describe an experimental target, not an accepted reliability standard. Our pilot does not meet both thresholds in either omission condition. Further work should identify conditions of success or reject the hypothesis rather than tune a demonstration until it appears successful.

2. Relationship to existing work

Dependency tracking and incremental computation are established research topics. Mokhov, Mitchell, and Peyton Jones [1] develop an executable framework for comparing build systems. Coetzee, Bhaskar, and Necula [2] discuss how manual dependency specifications can produce invalid outputs and outline a resource-oriented model for reliable builds. Lyu and colleagues [3] investigate dependency-error detection for incremental C/C++ builds using EChecker. Acar, Blume, and Donham [4] give a semantics and correctness treatment for self-adjusting computation.

We do not claim to invent incremental computation, dependency closure, sampled validation, or correctness under complete dependency tracking. This pilot contributes a small reproducible fault-injection harness and explicit accounting for the work of fresh audits and fallback. A comprehensive literature review and comparison against real dependency-tracking systems remain necessary before making a novelty claim. Unlike EChecker, our benchmark does not infer dependencies from program text or evaluate real build histories. Unlike a formal semantics for self-adjusting computation, it does not establish correctness for dynamic or effectful programs.

3. Model and restricted correctness argument

Let G=(V,E) be a finite directed acyclic graph with a fixed topological order. Every node is a deterministic task. An edge u -> v means that v reads u's output. A task output is a SHA-256 digest of its node identifier, local input salt, and ordered parent outputs. Dependencies and node functions remain fixed between runs; one local input salt changes. The declared graph D is a subgraph of G and may omit edges.

The full-replay reference evaluates every node in topological order. The transitive policy reevaluates the changed node and every node reachable from it in D. Other nodes reuse their previous outputs. Correctness means equality of the entire vector of task outputs with full replay, rather than equality only at designated sink nodes. An incorrect run contains at least one unequal output.

Proposition (restricted reuse correctness). If D=G, all task functions are deterministic and unchanged, all local input changes are included in a set C, and there are no unrepresented external inputs, reevaluating C and its descendants yields the same output vector as full replay.

Argument. Proceed through the topological order. A node outside the affected set has unchanged local input and no affected ancestor. Its parents and therefore its output are unchanged. A node inside the affected set is reevaluated after its parents, whose values agree with full replay by induction. Applying the same deterministic function yields the same value. This argument does not require hashing; it follows from the stated data dependencies. It is a standard dependency argument included to define the benchmark's boundary, not a claimed new theorem.

If D omits an edge, the argument can fail. In a three-node chain a -> b -> c, changing a while failing to invalidate c can retain an old result even if b is rerun. External state, clocks, randomness, network responses, changing task code, and effects must be represented or controlled before the proposition applies to a real system. A sampled audit provides no deterministic guarantee of detecting all violations.

4. Experiment design

The pilot uses 40 seeds (0 through 39), two sizes (64 and 256 tasks), two maximum fan-in values (1 and 3), and three edge-omission probabilities (0%, 5%, and 20%). This gives 480 scenarios, each evaluated under five policies. For node i, actual parents are uniformly sampled without replacement from preceding node identifiers, with count min(i, fan-in). This is an ordered random DAG model, not a uniform sample of all DAGs. Early nodes consequently have fewer parents. The changed node is selected uniformly from the first quarter of nodes, deliberately increasing potential downstream impact.

Each scenario first computes an all-zero-salt baseline, changes one salt to one, and computes a fresh reference. A separate seeded random stream independently drops edges from the declared graph. Reusing the same streams across omission probabilities couples the scenarios and produces nested declared edge sets. Seeds are also reused across sizes and fan-in settings. Accordingly, the 160 scenarios per omission condition must not be treated as 160 independent real-world observations.

The five policies are: full replay; direct-only invalidation (changed nodes and immediate declared children); transitive declared invalidation; and transitive invalidation followed by either a 2% or 10% cached-node audit. Direct-only invalidation is an intentionally weak diagnostic control, not a serious replacement for established incremental systems.

An audit uniformly samples ceil(q times cached-node count) cached nodes without replacement. It freshly evaluates each sampled node and its actual ancestors, memoizing this fresh context independently of the candidate execution. If a mismatch is found, it stops sampling and finishes full replay in that fresh context. On no mismatch, it returns the candidate output vector. The simulator exposes actual parent reads during fresh execution; a practical implementation would need equivalent instrumentation or isolated task execution. It does not let the selective scheduler see actual edges beforehand.

Work is the number of task evaluations divided by task count in full replay. Audit work includes all fresh ancestor evaluations. If a task was run during both selective execution and the fresh audit, both executions count. A fallback reuses work already completed in the fresh audit context, but not the candidate context. Values above 1.0 are possible. Graph traversal, storage, instrumentation, comparison, and scheduling costs are excluded; these are task-count results, not measured latency or energy savings. Reference construction is evaluation infrastructure and is excluded equally for all policies.

5. Pilot results

The table pools the four size/fan-in combinations equally, with 160 scenarios per omission condition. Each work ratio is calculated within a scenario before averaging. An incorrect-run percentage describes this finite benchmark grid, not an estimated production failure probability.

Missing edges	Policy	Incorrect runs / 160	Mean work / full
0%	Full	0	1.000
0%	Direct only	137	0.056
0%	Transitive	0	0.363

Missing edges	Policy	Incorrect runs / 160	Mean work / full
0%	Audit 2%	0	0.470
0%	Audit 10%	0	0.588
5%	Full	0	1.000
5%	Direct only	140	0.054
5%	Transitive	74	0.342
5%	Audit 2%	58	0.540
5%	Audit 10%	46	0.707
20%	Full	0	1.000
20%	Direct only	143	0.046
20%	Transitive	125	0.263
20%	Audit 2%	84	0.611
20%	Audit 10%	47	0.880

With complete dependencies, transitive invalidation requires 63.7% fewer task evaluations on average than full replay. The zero mismatches are consistent with the restricted argument; the theoretical conditions, not the finite test count, justify that argument. Direct-only invalidation often fails despite its small task count, illustrating why cost cannot be interpreted alone.

At 5% omission, the 10% audit removes 28 of the 74 incorrect runs remaining after transitive invalidation, a 37.8% reduction. This falls short of the proposed 50% target, although the mean work ratio remains below 0.75. At 20% omission, it removes 78 of 125 incorrect runs, a 62.4% reduction, but exceeds the work target. Even there, 47 of 160 runs remain incorrect. Thus neither nonzero omission condition satisfies both targets. No hypothesis tests, confidence intervals, or production reliability claims are inferred from these exploratory pooled counts.

6. Threats to validity

Hash-based tasks are highly change-sensitive: a parent change almost always changes downstream output. Real programs may normalize, ignore, or cancel inputs. This makes the simulator useful as a propagation stress test but a poor direct model of semantic equivalence. Cryptographic collision probability is not analyzed empirically. All tasks have unit cost, while real costs can differ by orders of magnitude. No cycles, concurrency, retries, failures, changing topology, incremental file reads, persistent side effects, or nondeterministic tasks are modeled.

Uniform independent omission does not capture structured failures such as an unrecorded environment variable used across many tasks. The restriction to early changed nodes biases the effect distribution. Complete access to actual dependencies during fresh audit is a substantial implementation assumption. Recursively rerunning ancestors may be infeasible for external services or unsafe for effectful tasks.

Auditing must not repeat irreversible actions. The benchmark validates neither sandboxing nor security against malicious workloads.

Pooling obscures size and topology differences; raw rows preserve those strata. Shared seeds create paired observations. No external replication has occurred. The implementation and manuscript were developed together with AI assistance, and the pilot was examined before the prospective protocol below was written. This study is not preregistered. The narrow related-work review does not establish originality.

7. Prospective validation and engineering work

Freeze the current pilot and protocol before a separate validation run. Use unseen seeds 40 through 239; sizes 128 and 512; fan-in 1, 3, and 6; and omission probabilities 1%, 5%, and 20%. Include both independent omission and a documented clustered-omission mechanism. Select changed nodes from early, middle, and late graph regions in separate strata. Add heterogeneous task costs and workloads with semantically unchanged outputs.

The primary endpoint should test the 10% audit at 5% independent omission: at least a 50% reduction in incorrect runs relative to transitive invalidation and mean total task-work ratio at most 0.75. Freeze stratum weights before running. Treat seed as the resampling cluster for paired uncertainty intervals; report the correctness difference and work difference as well as the joint target. Clustered omissions, alternative audit budgets, mutation positions, and other rates are secondary analyses. Report failed targets without redefining them after results are seen.

Before practical claims, implement read tracking for a small local, side-effect-free workflow runner, replay real edit histories, and compare against full execution and an established incremental baseline. Measure wall time and metadata overhead separately. Evaluate audit strategies against random sampling without assuming a new strategy is superior. Conduct a broader literature review and solicit independent reproduction. A publishable contribution may ultimately be a negative result about audit economics rather than a successful general-purpose replay method.

8. Reproducibility and availability

The companion repository is https://github.com/theworker02/replay-integrity-lab. It is initially private during author review. A public artifact snapshot and persistent identifier are prerequisites for an independently reproducible public preprint. No DOI has been issued for this draft.

Python 3.10 or later is sufficient for the experiment; it has no third-party dependencies. Run `python -m unittest discover -v` and `python experiment.py` from the repository root. The latter regenerates `results/trials.csv`, `results/summary.json`, and `results/manifest.json`. Six unit/regression tests cover transitive propagation, an audit-detected mismatch and fallback, an empty audit set, complete-dependency consistency over additional seeds, deterministic repetition, and the full reference under omissions. Tests do not substitute for independent implementation or real-workload evaluation.

9. Conclusion

Selective replay is correct within a restricted deterministic model when all dependencies and input changes are represented. Incomplete dependencies break that guarantee. In this pilot, sparse fresh-execution audits reduce incorrect runs but leave substantial residual error and can approach full-replay work. The proposed cost-and-correctness hypothesis remains unresolved beyond this model and fails its joint target in the two nonzero-omission pilot conditions. The immediate research task is to validate or reject useful workload-specific regimes, then test an instrumented implementation on real histories.

Declarations

Author: Matthew Looney, Magnexis. ORCID: 0009-0007-0025-8000. AI assistance: OpenAI Codex assisted with study design, implementation, execution, literature discovery, and drafting. Author review of references, claims, licenses, and the final manuscript is pending. No claim of independent human verification is made. No human-subject data were collected. Commercial interest: the author develops software through Magnexis, which may benefit from workflow research. No external funding is reported in this draft; the author should confirm that statement before submission. This manuscript is a working preprint draft, not an accepted or peer-reviewed publication.

References

[1] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems a la Carte. Proceedings of the ACM on Programming Languages, ICFP. https://www.microsoft.com/en-us/research/publication/build-systems-la-carte/

[2] Derrick Coetzee, Anand Bhaskar, and George Necula. 2012. A model and framework for reliable build systems. arXiv:1203.2704. https://arxiv.org/abs/1203.2704

[3] Jun Lyu, Shanshan Li, He Zhang, Yang Zhang, Guoping Rong, and Manuel Rigger. 2024. Detecting Build Dependency Errors in Incremental Builds. arXiv:2404.13295. https://arxiv.org/abs/2404.13295

[4] Umut A. Acar, Matthias Blume, and Jacob Donham. 2011. A Consistent Semantics of Self-Adjusting Computation. arXiv:1106.0478. https://arxiv.org/abs/1106.0478